A01:2025 Broken Access Control

A01:2025 Broken Access Control：實戰程式碼解析

從「隱式授權」陷阱到「企業級」安全設計

針對開發者與資安工程師設計的實戰指南。我們將透過一段真實的 PHP/SQL 程式碼，剖析看似安全的寫法為何會導致嚴重漏洞，並示範符合企業標準的修正方案。

JetBrains Mono: OWASP Top 10 Security Guide // Developer Edition

A01:2025 Broken Access Control

看似安全的陷阱：隱式授權（Implicit Authorization）

當權限判斷「寄生」於資料查詢時，風險隨之而來

A01

```php
$userId = $_SESSION['user_id'];
$accountId = $_GET['id'];

//     風險設計：將權限判斷寄生於 SQL 條件
// 試圖透過 AND owner_user_id = ? 來防止他人存取
$stmt = $pdo->prepare("SELECT * FROM accounts WHERE id = ? AND owner_user_id = ?");
$stmt->execute([$accountId, $userId]);
$row = $stmt->fetch();

if (!$row) {
    // 模糊的回應：無法區分是「無資料」還是「無權限」
    http_response_code(403);
}
```

這段程式碼雖然防住了 SQL Injection，卻留下了嚴重的 Access Control 漏洞。

A01:2025 Broken Access Control

為什麼這樣寫是錯的?(The Analysis)

1. 邏輯寄生

將「誰能看」藏在「怎麼查」裡面。
一旦 SQL 被修改（例如複製去別處使用並移除了 owner_user_id 條件），IDOR漏洞立刻復活。

2. 角色缺失

系統假設「只有 Owner 能看」若未來加入 Admin、客服或稽核人員（Auditor）角色，這段 SQL 必須重寫，否則 Admin 也查不到資料。

3. 資訊洩漏

系統回應無法區分：
「資源不存在（404）」
「沒有權限（403）」
攻擊者可藉此枚舉資料庫內容（推測哪些 ID 存在）。

A01:2025 Broken Access Control

正確修正：建立明確的授權層（Explicit Authorization）

Step 1: Data Access

```php
// 步驟 1：先查資源
$stmt = $pdo->prepare("SELECT * FROM accounts WHERE id = ?");
$stmt->execute([$accountId]);
$account = $stmt->fetch();

if (!$account) {
    http_response_code(404); // [redacted] 正確告知資源不存在
    exit;
}
```

Step 2: Authorization Layer

```php
// 步驟 2：再查權限
// [redacted] 集中管理邏輯：允許 Admin 或 Owner 存取
$role = $_SESSION['role'] ?? 'user';
if ($role !== 'admin' && $account['owner_user_id'] !== $userId) {
    http_response_code(403); // [redacted] 明確告知無權限
    exit;
}
echo json_encode($account);
```

優勢：
邏輯集中、可審計
支援多重角色
（Admin / Support）
避免 Copy-Paste 漏洞

A01:2025 Broken Access Control

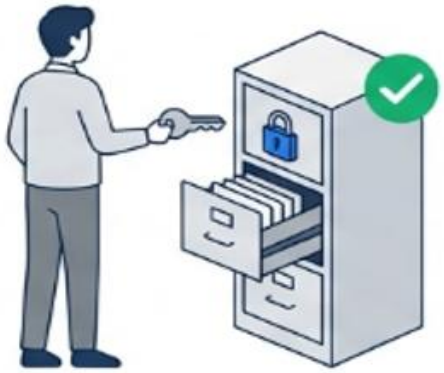
觀念釐清：AuthN 與 AuthZ 的區別

Authentication (AuthN)

你是誰？

機制：驗證帳號 / 密碼、JWT Token 、Session ID
常見誤區：
「使用者已登入，所以安全。」

Authorization (AuthZ)



你能做這件事嗎？

機制：檢查權限規則、物件級存取控制
A01 核心：
每一筆 Request 都要檢查。

擁有「進大樓的識別證」（AuthN），不代表能進入「金庫」（AuthZ）。

A01:2025 Broken Access Control

A01 防禦重點清單

1.不再依賴 SQL WHERE 做權限控制

將資料查詢（Data Access）與權限檢查（AuthZ）邏輯拆開，避免隱式授權。

2.實施物件級存取控制

不要以為有 Token 就有權限，必須檢查 user_id == resource_owner。

3.預設拒絕

除了明確允許的人（如 Admin / Owner），其他人一律回應 403。

4. 避免資訊洩露

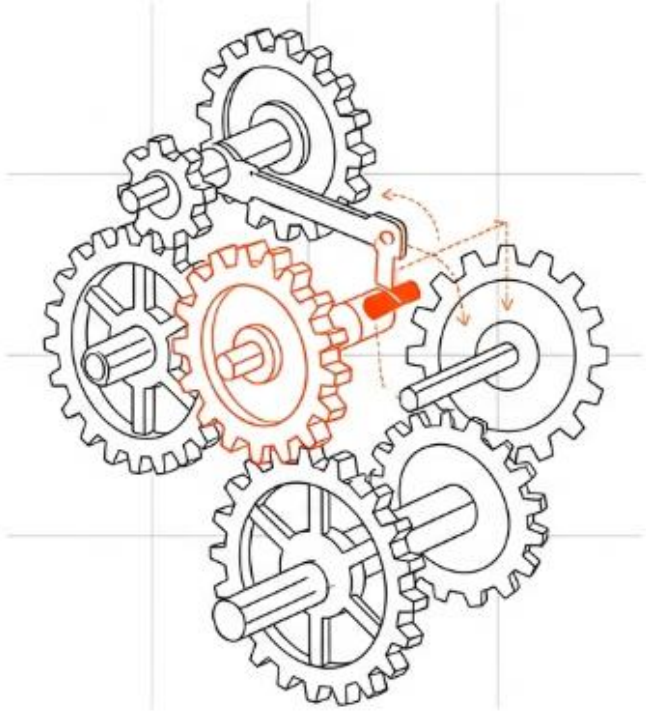
正確區分並使用 404（找不到）與 403（禁止），防止攻擊者枚舉資源。

5.集中化授權管理

在大型專案中，將授權邏輯抽離為 Middleware 或 Policy，避免個別 API 遺漏檢查。

A02:2025 Security Misconfiguration

A02:2025
安全設定失誤
給研發團隊的防禦指南



OWASP Top 10 2025

A02:2025 Security Misconfiguration

這是最普遍的風險

在測試的應用程式中，發現某種形式設定失誤的比例

A02:2025 Security Misconfiguration

什麼是安全設定失誤？

系統、應用程式或雲端服務的設定從安全角度來看是不正確的

不安全的預設值

預設帳號與密碼未更改，
啟用不必要的功能或範例應用程式。

資訊洩露

錯誤處理不當，
向使用者揭露 **Stack Traces** 或詳細錯誤訊息。

平台暴露

雲端權限（如 S3 buckets）設定過於寬鬆，
未安裝安全修補程式。

A02:2025 Security Misconfiguration

攻擊案例分析：伺服器端

配置失誤
管理後臺使用試射帳密

攻擊後果
攻擊者登入並接管伺服器

配置失誤
目錄列表功能位關閉

攻擊後果
攻擊者下載編譯後的Java Classes，
反編譯並發現存取控制漏洞

A02:2025 Security Misconfiguration

攻擊案例分析：數據與雲端

配置失誤
應用程式回傳詳細的錯誤訊息

```
HTTP 500 Internal Server Error
...
at con.example.app.Controller.
handle(Controller.java:123)
at org.springframework.
web.servlet.DispatcherServlet.
.doDispatch(DispatcherServlet.
java:980)
... more
```

攻擊後果
暴露組件版本與已知漏洞，協助攻擊者精準打擊

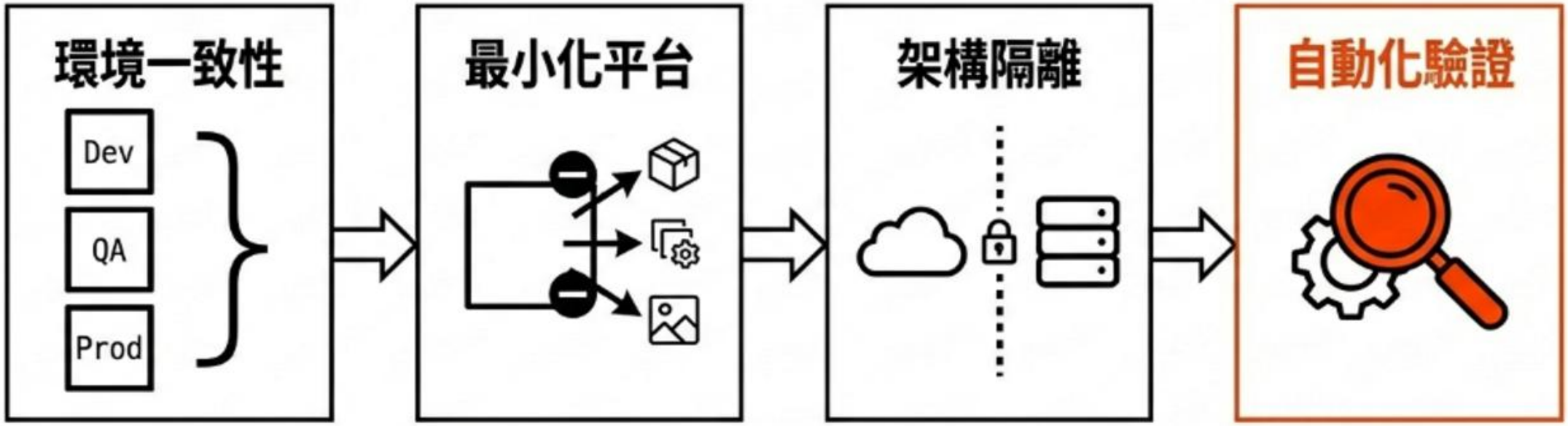
配置錯誤
雲端儲存(如S3)預設對網際網路開放

攻擊後果
敏感數據直接外洩

A02:2025 Security Misconfiguration

防禦策略：架構與流程

建立可重複且自動化的加固流程



開發、QA 和生產環境的配置應完全相同

移除位使用的功能、框架及範例程式

使用容器化或雲端安全群組（ACLs）來隔離組件

自動檢查所有環境的配置有效性

A02:2025 Security Misconfiguration

防禦策略：實作細節

安全標頭

向客戶端發送安全指令（如 HSTS、CSP、X-Frame-Options）

統一錯誤處理

全局攔截 Stack Traces，
避免向使用者顯示過多資訊。

憑證管理

使用短期憑證或 IAM 角色，
嚴禁將靜態金鑰寫入程式碼。

雲端權限審查

定期審查 S3 Bucket 與儲存權限。

A02:2025 Security Misconfiguration

開發者重點清單

- **拒絕預設值：** 更改所有預設帳號與密碼
- **最小化安裝：** 移除不必要的功能與範例
- **隱藏細節：** 確保錯誤訊息不包含 Stack Traces
- **自動化審查：** 將配置檢查納入 CI/CD

A02:2025 Security Misconfiguration

```php
<?php
ini_set('display_errors', '1'); // ❌ production 開啟
error_reporting(E_ALL);

try {
    // 模擬一個例外
    throw new Exception("DB connection failed: host=10.0.0.5 user=app");
} catch (Exception $e) {
    http_response_code(500);
    echo $e; // ❌ 直接輸出例外（可能含路徑/堆疊/敏感資訊）
}
```

```php
<?php
ini_set('display_errors', '0'); // ✅ production 關閉
error_reporting(E_ALL);

try {
    throw new Exception("DB connection failed: host=10.0.0.5 user=app");
} catch (Exception $e) {
    error_log($e); // ✅ 詳細資訊留在伺服器端
    http_response_code(500);
    header('Content-Type: application/json');
    echo json_encode(['error' => 'Internal Server Error']); // ✅ 對外只給通用訊息
}
```

A02:2025 Security Misconfiguration

```php
<?php
header('Access-Control-Allow-Origin: *');          //  任何網站都能呼叫
header('Access-Control-Allow-Credentials: true'); //  還允許帶 cookie/token
header('Content-Type: application/json');

echo json_encode(['data' => 'sensitive']);
```

```php
<?php
$allowed = ['https://app.example.com'];

$origin = $_SERVER['HTTP_ORIGIN'] ?? '';
if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: $origin");
    header('Access-Control-Allow-Credentials: true');
}

header('Content-Type: application/json');
echo json_encode(['data' => 'sensitive']);
```